

Proiect

Dispozitive Electronice si Electronica Analogica

Anul II – Semestrul I



Smart Home

Dobre Mario-Sebastian

324CA

1. Descrierea Generală a Proiectului

1.1 Funcționalitate și Scenariu de Utilizare

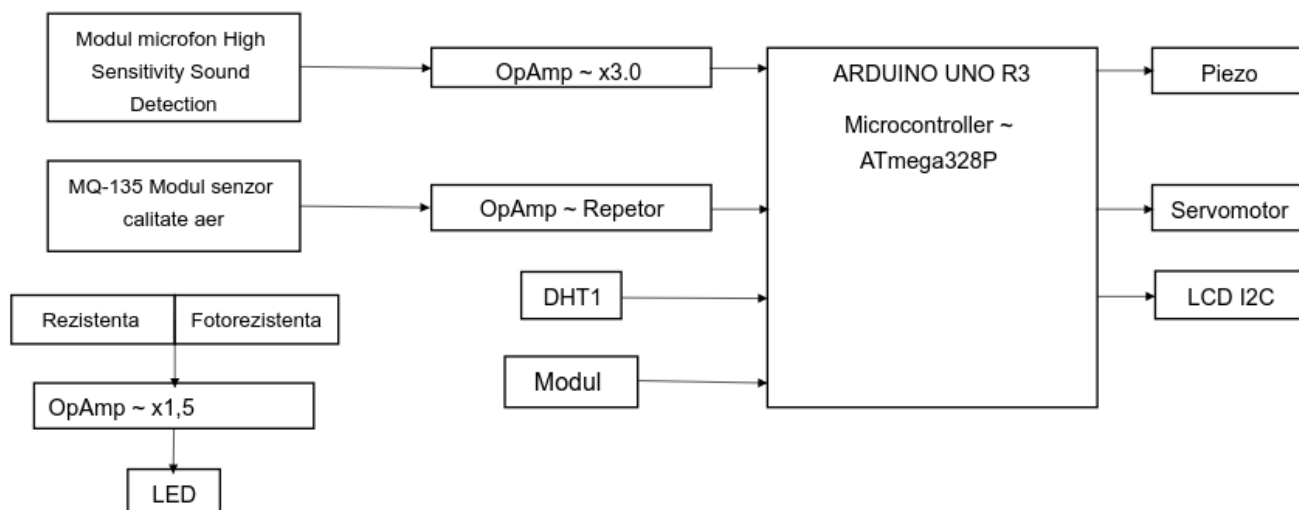
Acest proiect propune implementarea unei arhitecturi hardware care prelucrează semnale de tip analogic și digital, destinată monitorizării și controlului automatizat al parametrilor de siguranță și confort dintr-o locuință inteligentă.

Sistemul integrează două tipuri de lanțuri de colectare a datelor:

1. Lanțul de procesare analogică: Mărimile fizice continue: sunet, lumină și calitatea aerului, sunt preluate de senzorii specifici și trecute printr-un etaj realizat cu Amplificatoare Operaționale. Aceste circuite au rolul de a amplifica și adapta impedanța semnalului înainte ca datele să fie colectate de microcontroller.
2. Lanțul de procesare digitală: Este utilizat pentru monitorizarea parametrilor care necesită o precizie ridicată sau calibrare specifică. Se utilizează senzori: DHT11 pentru temperatură/umiditate și PIR pentru mișcare, care prelucrează intern informația și o transmit către microcontroller.

Pe baza acestor date, microcontroller gestionează logica de decizie în timp real, acționând actuatori: servomotor, buzzer, afișaj, pentru a asigura protecția și informarea utilizatorului.

1.2 Schema Bloc



2. Lista de Materiale

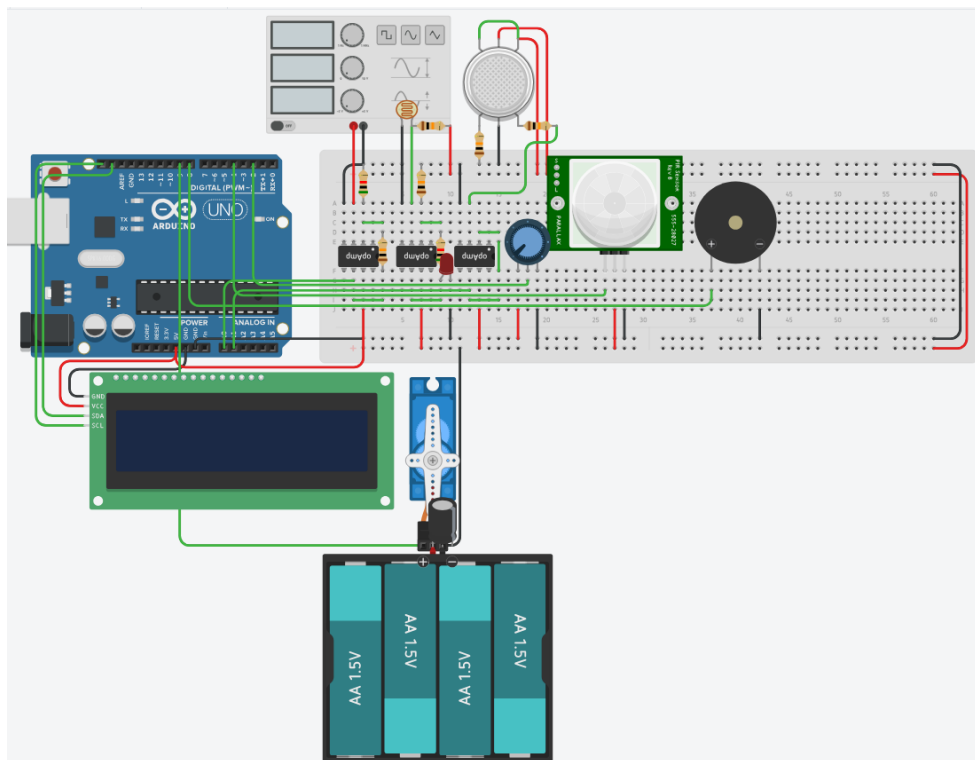
Nr. Crt.	Componentă	Tip	Cant.	Rol în circuit
1.	Placă de Dezvoltare	Arduino UNO R3	1	Unitate centrală de procesare.
2.	Conectică	Breadboard + Fire	1	Realizarea conexiunilor fizice.
3.	Sursă Alimentare	Baterii 4xAA	1	Alimentare externă pentru stabilitate.
4.	Amplificator Operațional	TL072CP	2	Circuite integrate pentru condiționarea semnalelor analogice (amplificare și repetare)
5.	Senzor Temperatură/Umiditate	DHT11	1	Monitorizare parametri ambientali (Digital).
6.	Senzor de Mișcare	Modul PIR	1	Detector de prezență bazat pe variația radiației infraroșii
7.	Microfon	Modul microfon High Sensitivity Sound Detection	1	Traductor acustic pentru detecția zgomotului.
8.	Senzor calitate aer	MQ-135	1	Traductor rezistiv pentru detectarea concentrației gazelor
9.	Fotorezistență	90mW	1	Senzor analogic pentru luminozitate.
10.	Servomotor	SM-S2309S	1	Mecanism de închidere automată de urgență a vanei de gaz.
11.	Afișaj LCD	16x2 cu modul I2C	1	Interfață vizuală pentru utilizator
12.	Buzzer	Piezo	1	Avertizare sonoră în caz de alarmă.
13.	Rezistente	5kΩ, 10kΩ,	Set	Divizoare de tensiune și rețea de reacție OpAmp.
14.	Condensator	100μF	1	Inmagazinare curent pentru servomotor

3. Proiectarea și Simularea Circuitului (TinkerCad)

Pentru a valida conceptul înainte de realizarea fizică, arhitectura a fost implementată inițial în TinkerCad. Această etapă a avut ca scop verificarea conexiunilor electrice dintre senzorii analogici, etajele de amplificare cu AO și placa de dezvoltare, precum și testarea logicii de activare a actuatorilor.

3.1 Schema Electrică

Schema realizată în simulator reprezintă harta completă a conexiunilor sistemului, incluzând divizorul de tensiune și configurația pinilor de intrare/ieșire.



Notă: Din lipsa componentelor în TinkerCad, Microfonul a fost substituit cu un Generator de Funcții (pentru testarea amplificării), iar senzorul DHT11 cu un Potenționmetru (pentru simularea semnalului).

Fig. 3.1. Schema electrică completă simulată în TinkerCad.

4. Implementarea Fizică a Sistemului

Această secțiune reprezintă transpunerea schemei simulate într-un montaj real.

4.1 Realizarea Montajului pe Breadboard

Sistemul a fost asamblat pe un breadboard, respectând schema electrică validată anterior.

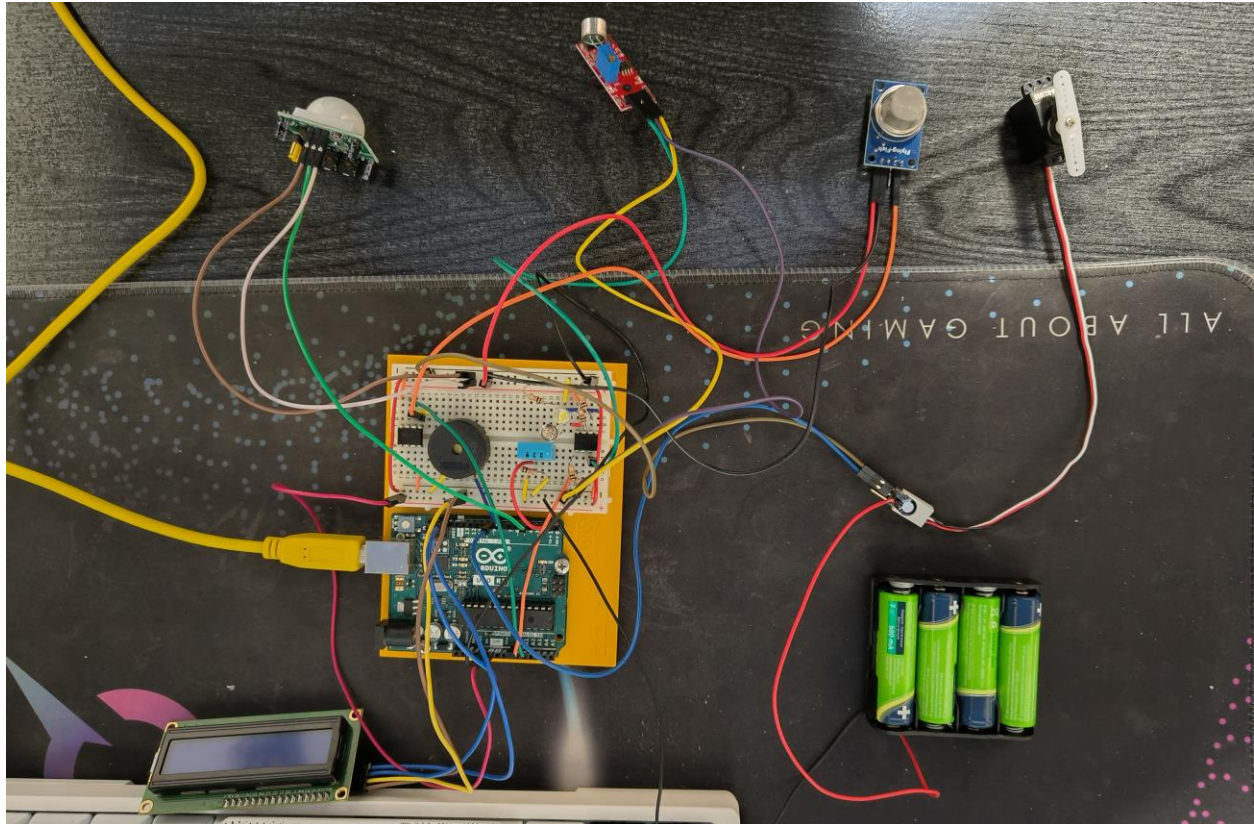


Fig. 4.1. Imagini ale montajului fizic experimental.

4.2 Demonstrație Funcțională

Sistemul a fost testat în scenarii reale pentru a demonstra atingerea obiectivelor proiectului:

1. **Detecție Zgomot:** Reacția amplificatorului la sunete ambientale (lovituri in sticla).
2. **Siguranță:** Acționarea servomotorului la simularea prezenței gazului.
3. **Monitorizare:** Afișarea corectă a temperaturii și umidității pe LCD.

4. **Aprindere LED:** Activarea automată a sursei de lumină la întuneric, realizată printr-un circuit pur analogic care acționează direct LED-ul.

Funcționalitatea completă este prezentată în materialul video atașat:

<https://drive.google.com/drive/folders/1vRINiasUKfXrKkUDrEzembGJobrwJ1HX>

5. Codul Sursă

Logica de funcționare a fost implementată în limbajul C++, folosind **Arduino IDE**.

Programul rulează într-o buclă infinită care interoghează senzorii în mod secvențial:

- **setup():** Inițializează pinii de intrare/ieșire, comunicația cu afișajul LCD și poziționează servomotorul în stare inițială (deschis).
- **loop():** Bucla principală a programului, care apelează ciclic funcțiile de procesare.
- **processGas():** Citește valoarea furnizată de senzorul analogic MQ-135. Dacă aceasta depășește pragul de siguranță stabilit, funcția acționează servomotorul pentru a simula închiderea vanei de gaz, care rămâne blocată până la intervenția utilizatorului.
- **processMicrophone():** Citește semnalul sonor, amplificat printr-un amplificator neinvertor. Dacă nivelul sonor iese din intervalul de calibrare, este declanșată alarma, care rămâne activă până la intervenția utilizatorului.
- **processPIR():** Verifică starea senzorului de mișcare și emite avertizări sonore scurte la detectarea unei mișcări.
- **processDHT11():** Citește valorile de temperatură și umiditate la intervale regulate și actualizează informațiile afișate pe ecranul LCD.
- **serialLog():** Transmite periodic valorile primite de la senzorii către interfața serială. Această funcție este esențială pentru mentenanță și diagnoză, permițând operatorului să identifice rapid eventuale derapaje ale senzorilor sau date eronate.

6. Direcții Viitoare

Pentru optimizarea performanțelor și transformarea prototipului într-un produs comercial, se propun următoarele îmbunătățiri majore:

1. **Filtrare Analogică:** Adăugarea unui etaj de filtrare (High-Pass Filter) în lanțul de procesare audio. Acesta ar permite eliminarea zgomotului de fond de joasă frecvență și izolarea frecvențelor înalte, specifice evenimentelor de efracție (sunetul de sticlă spartă), crescând precizia detecției.
2. **Optimizare Software prin Regiștri Hardware:** Rescrierea secvențelor critice de cod prin manipularea directă a regiștrilor microcontroller-ului ATmega328P, în locul utilizării funcțiilor standard Arduino.
3. **Transmitere de date și monitorizare la distanță:** Integrarea unui modul Wi-Fi pentru transmiterea periodică a datelor către un server extern sau o bază de date. Aceasta ar permite generarea unor grafice de evoluție în timp a calității aerului și temperaturii, oferind utilizatorului o imagine de ansamblu asupra istoricului locuinței.
4. **Sursă de Alimentare Unificată:** Înlocuirea soluției actuale (USB + baterii separate) cu un circuit integrat de management al energiei.

7. Bibliografie și Resurse Utilizate

În elaborarea acestui proiect, documentarea tehnică și implementarea au fost realizate consultând următoarele surse:

A. Resurse Academice:

1. **Cursul de Dispozitive Electronice și Electronică Analogică (Anul II):** Suport de curs și laboratoare (încărcate pe platforma facultății), utilizate pentru calculul etajelor de amplificare cu AO și înțelegerea circuitelor.

B. Documentație Tehnică (Datasheets):

1. **Texas Instruments**, *TL07xx Low-Noise, FET-Input Operational Amplifiers*
2. **Wiring**, *Air Quality MQ135 sensor by BARRAGAN*
3. **Arduino LLC**, *Arduino UNO R3 Reference Manual*
4. **Elecrow**, *HC-SR501 PIR Motion Sensor User Guide*.

C. Resurse Software și Biblioteci (GitHub/Arduino):

1. **DHT Sensor Library** (by Adafruit) – Sursă: <https://github.com/adafruit/DHT-sensor-library>
2. **LiquidCrystal I2C** (by Frank de Brabander) – Sursă: <https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library>
3. **Servo Library** (Standard Arduino) – Utilizată pentru generarea semnalului PWM.

Anexa 1

```
#include <LiquidCrystal_I2C.h>
#include <DHT.h>
#include <Servo.h>

#define MIC_PIN A0
#define GAS_PIN A1
#define DHT_PIN 2
#define BUZZER_PIN 8
#define PIR_PIN 4
#define SERVO_PIN 9
#define DHTTYPE DHT11

DHT dht(DHT_PIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo gasServo;

const int GAS_THRESHOLD = 330;
const int MIC_LOWER = 90;
const int MIC_UPPER = 110;
bool ALARM = false;
long micValue = 0;
int gasValue = 0;
int pirState = 0;
float temperature = 0;
float humidity = 0;

void setup() {
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(PIR_PIN, INPUT);

  Serial.begin(9600);
  dht.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();
  lcdPrintMessage("Sistem pornit", "");

  gasServo.attach(SERVO_PIN);
  gasServo.write(0);
}

void loop() {
```

```

    processGas();
    processMicrophone();
    processPIR();
    processDHT11();

    serialLog();

    delay(100);
}

void processGas() {
    gasValue = analogRead(GAS_PIN);
    if (gasValue > GAS_THRESHOLD) {
        gasServo.write(90);
    } else {
        gasServo.write(0);
    }
}

void processMicrophone() {
    micValue = analogRead(MIC_PIN);
    if (micValue < MIC_LOWER || micValue > MIC_UPPER) {
        ALARM = true;
        tone(BUZZER_PIN, 1000);
    }
}

void processPIR() {
    pirState = digitalRead(PIR_PIN);
    if (pirState == HIGH && !ALARM) {
        tone(BUZZER_PIN, 100);
    } else if (!ALARM) {
        noTone(BUZZER_PIN);
    }
}

void processDHT11() {
    temperature = dht.readTemperature();
    humidity = dht.readHumidity();
    if (!isnan(temperature) && !isnan(humidity)) {
        lcdPrintMessage(
            "Temp: " + String(temperature, 1) + (char)223 + "C",
            "Umid: " + String(humidity, 1) + "%"
        );
    }
}

```

```

void lcdPrintMessage(String line1, String line2) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print(line1);
    lcd.setCursor(0, 1);
    lcd.print(line2);
}

void serialLog() {
    Serial.println("----- CITIRI SENZORI -----");
    Serial.print("Microfon (A0): "); Serial.println(micValue);
    Serial.print("Gaz (MQ135 A1): "); Serial.println(gasValue);
    Serial.print("PIR (D4): "); Serial.println(pirState == HIGH ? "DETECTIE" :
"INACTIV");
    Serial.print("Temperatura (°C): "); Serial.println(temperature, 1);
    Serial.print("Umiditate (%): "); Serial.println(humidity, 1);
    Serial.println("-----\n");
}

```